

Lista de Atividade prova 3 - Aeds 02

Estruturas de Dados e Algoritmos II

Feito por: Gabriel Ferreira Pereira

Balanceamento

1) Implementação em árvore binária com critério de AVL

Considere uma estrutura de dados representada por uma árvore binária não balanceada, implementada pela classe *Arvore*. A árvore é composta por nós, representados pela classe *No*, conforme a estrutura abaixo.

Implemente, em Java, o método `contarNosNaoBalanceados()` na classe *Arvore*. Esse método deve retornar um número inteiro que representa a quantidade de nós da árvore cujo fator de balanceamento não atende ao critério de uma árvore AVL.

Não é permitido adicionar novos atributos às classes existentes. Faça também a análise de complexidade da sua solução.

```
class Arvore {
    private No raiz;

    public int contarNosNaoBalanceados() {
        // IMPLEMENTAR
        return 0;
    }
}

class No {
    public int elemento;
    public No esq, dir;
}
```

2) Inserções passo a passo em AVL, 2-3-4 e alvinegra

Considere a sequência de números:

14, 9, 22, 5, 11, 18, 27, 3, 7, 10, 13, 16, 20

Construa a estrutura resultante, após a inserção de cada número da sequência, para cada uma das árvores descritas a seguir:

1. uma árvore AVL, ajustando os balanceamentos sempre que necessário após cada inserção;
2. uma árvore 2-3-4, utilizando o método de fragmentação na descida;
3. uma árvore alvinegra, aplicando as rotações e recolorações apropriadas para manter as propriedades da estrutura.

Apresente o esboço de cada árvore ao final de cada inserção.

3) Provar ou refutar afirmações sobre AVL, 2-3-4 e alvinegra

Analise as afirmações a seguir e determine se cada uma é verdadeira ou falsa, apresentando uma justificativa detalhada.

1. Considerando árvores AVL e alvinegras, AVL são preferidas quando as operações de busca são mais frequentes que inserções e remoções.
2. Em uma árvore 2-3-4, a fragmentação na descida evita que a recursão de retorno precise tratar estouros de nós.
3. Uma rotação dupla direita-esquerda em AVL ocorre quando o pai está pesado à direita e o filho direito está pesado à esquerda.
4. A árvore alvinegra pode ser vista como uma representação binária equivalente de árvores 2-3-4.

4) Implementar balanceamento local em AVL

Implemente o método `balancear(No no)` de uma árvore AVL. Considere que as rotações já existem. O método deve examinar o fator de balanceamento do nó e de seu filho apropriado para decidir entre rotação simples e dupla. Faça a análise de complexidade.

```
class No {
    int elemento, altura;
    No esq, dir;
}

class AVL {
    private int getAltura(No no) {
        return (no == null) ? -1 : no.altura;
    }

    private int fator(No no) {
        return (no == null) ? 0 : getAltura(no.dir) - getAltura(no.esq);
    }

    private No rotacionarEsq(No no) { return no; }
    private No rotacionarDir(No no) { return no; }

    public No balancear(No no) {
        // IMPLEMENTAR
        return no;
    }
}
```

Hash

5) Inserção em tabela hash com reserva

Implemente o método `inserir(String s)` em uma tabela hash que usa área primária e uma área de reserva linear para tratar colisões. A função hash é a soma dos códigos ASCII dos

caracteres módulo m_1 . Faça a análise de complexidade.

```
class HashReserva {
    String[] tabela;
    int m1, m2, reserva;

    public HashReserva(int m1, int m2) {
        this.m1 = m1;
        this.m2 = m2;
        this.tabela = new String[m1 + m2];
        this.reserva = 0;
    }

    private int h(String s) {
        int soma = 0;
        for (int i = 0; i < s.length(); i++) soma += s.charAt(i);
        return soma % m1;
    }

    public boolean inserir(String s) {
        // IMPLEMENTAR
        return false;
    }
}
```

6) Pesquisa em hash com rehash linear

Implemente o método `pesquisar(int x)` em uma tabela hash aberta com rehash linear. Considere que posições vazias guardam `-1`. Faça a análise de complexidade média e de pior caso.

```
class HashLinear {
    int[] tabela;
    int m;

    private int h(int x) {
        return x % m;
    }

    public boolean pesquisar(int x) {
        // IMPLEMENTAR
        return false;
    }
}
```

7) Provar ou refutar afirmações sobre hash

Analise as afirmações abaixo e classifique cada uma como verdadeira ou falsa, justificando detalhadamente.

1. Uma boa função hash deve espalhar uniformemente as chaves pelas posições da tabela.
2. Endereçamento aberto sempre usa menos memória do que encadeamento externo em qualquer cenário.
3. Uma estrutura hash é, em geral, mais adequada para busca exata do que para busca por prefixo.
4. Em tabelas com rehash linear, o agrupamento primário pode degradar o desempenho das buscas.

Doidona e Estruturas Híbridas

8) Pesquisa na estrutura Doidona

Considere uma estrutura de dados **Doidona** cujo primeiro nível contém uma árvore binária de busca com 26 nós, um para cada letra inicial. Cada nó aponta para uma tabela T1. Em T1, a palavra é distribuída pelo último caractere. Quando T1 não consegue resolver a colisão, a palavra é encaminhada para T2; quando T2 não consegue resolver a colisão, a palavra vai para T3, que pode encaminhar para uma árvore binária ou para uma lista, de acordo com a paridade do valor retornado por `hashT3`.

Implemente o método `boolean pesquisar(String palavra)` na classe **Doidona**. A pesquisa deve explorar a árvore de letras, a tabela T1, a tabela T2 e, se necessário, a estrutura T3. Faça a análise de complexidade.

```
class Doidona {
    private No raiz;

    public boolean pesquisar(String palavra) {
        // IMPLEMENTAR
        return false;
    }
}

class No {
    public char caractere;
    public T1 t1;
    public No esq, dir;
}

class T1 {
    public String[] palavras;
    public T2 t2;
}

class T2 {
    public String[] tabela;
    public T3 t3;
}

class T3 {
```

```

    public NoAB raiz;
    public Celula primeiro;
    public int hashT3(int x) { return x % 2; }
}

class NoAB {
    int elemento;
    NoAB esq, dir;
}

class Celula {
    String palavra;
    Celula prox;
}

```

9) Inserção em estrutura híbrida árvore + hash + lista

Considere uma estrutura de dados **DicionarioHibrido**. O primeiro nível contém uma árvore binária de busca composta por letras do alfabeto, usada para localizar palavras pelo primeiro caractere. Cada nó dessa árvore possui uma referência para uma tabela hash T1. A tabela T1 possui tamanho m1 e distribui as palavras usando o último caractere. Caso a posição calculada em T1 já esteja ocupada, deve-se tentar um único rehash. Se ainda assim houver colisão, a palavra deve ser enviada para uma estrutura T2, implementada como um vetor de listas encadeadas, indexado pelo tamanho da palavra módulo m2.

Implemente o método `void inserir(String palavra)` na classe **DicionarioHibrido**. Faça a análise de complexidade da inserção.

```

class DicionarioHibrido {
    private No raiz;

    public void inserir(String palavra) {
        // IMPLEMENTAR
    }
}

class No {
    public char letra;
    public T1 t1;
    public No esq, dir;
}

class T1 {
    public String[] tabela;
    public T2 t2;
    public int hashT1(char x) { return x % m1; }
    public int rehashT1(char x) { return (++x) % m1; }
}

class T2 {

```

```

    public CelulaLista[] tabela;
    public int hashT2(int tamPalavra) { return tamPalavra % m2; }
}

class CelulaLista {
    public String palavra;
    public CelulaLista prox;
}

```

Trie

10) Interseção entre prefixo e sufixo em trie

Considere uma árvore trie que aceita prefixos compartilhados, implementada pela classe `ArvoreTrie`. Cada nó representa um caractere, e os caminhos da raiz até os nós com `fimPalavra = true` correspondem a palavras válidas. Os filhos de um nó são armazenados em uma lista encadeada com célula cabeça.

Implemente, em Java, o método `boolean existe(String inicio, String fim)` na classe `ArvoreTrie`. Esse método deve retornar `true` se houver pelo menos uma palavra completa na árvore que comece com a string `inicio` e termine com a string `fim`, e `false` caso contrário.

Faça também a análise de complexidade da solução.

```

class ArvoreTrie {
    No raiz;

    public boolean existe(String inicio, String fim) {
        // IMPLEMENTAR
        return false;
    }
}

class No {
    char elemento;
    Celula primeiro; // clula cabea
    Celula ultimo;
    boolean fimPalavra;
}

class Celula {
    char elemento;
    Celula prox;
    No no;
}

```

11) Listar palavras com um dado prefixo

Implemente o método `mostrarComPrefixo(String pref)` que imprime todas as palavras armazenadas em uma trie que começam com o prefixo informado. Primeiro localize o nó do prefixo e, em seguida, percorra a subárvore correspondente. Faça a análise de complexidade.

```
class Trie {
    private No raiz;

    public void mostrarComPrefixo(String pref) {
        // IMPLEMENTAR
    }
}
```

Patricia

12) Projeto de nó para trie Patricia

O nó de uma árvore trie tem um custo extra para gerenciar seus filhos. Por isso, uma trie Patricia costuma ser usada para compactar caminhos unitários.

Apresente e justifique três alternativas de implementação para os filhos de um nó de uma trie Patricia:

1. vetor indexado diretamente por caractere;
2. árvore binária de busca entre os filhos;
3. tabela hash entre os filhos.

Em seguida, escolha uma das três estratégias e implemente a classe `NoPatricia`, incluindo os atributos necessários para representar:

1. o rótulo comprimido do arco;
2. a marcação de fim de palavra;
3. a estrutura usada para os filhos.

Justifique sua escolha em termos de busca por prefixo, uso de memória e custo esperado das operações.

```
class NoPatricia {
    // IMPLEMENTAR
}
```

13) Provar ou refutar afirmações sobre trie e Patricia

Analise as afirmações abaixo e classifique cada uma como verdadeira ou falsa, justificando detalhadamente.

1. Toda trie Patricia representa o mesmo conjunto de palavras de uma trie comum equivalente.

2. A trie Patricia elimina caminhos internos redundantes formados por nós com um único filho.
3. Uma trie comum tende a facilitar buscas por prefixo.
4. Se o conjunto de palavras compartilha muitos prefixos longos, a Patricia tende a economizar espaço em relação à trie comum.